

Technical Section

Image steganalysis method based on cover selection and adaptive filtered residual network[☆]Yuanyuan Ma^{a,b}, Zenghao Yang^{a,*}, Tao Li^{a,b}, Lige Xu^{a,b}, Yaqiong Qiao^{c,d}^a College of Computer and Information Engineering, Henan Normal University, Xinxiang 453007, China^b Key Laboratory of Artificial Intelligence and Personalized Learning in Education of Henan Province, Xinxiang, China^c School of Information Engineering, North China University of Water Resources and Electric Power, Zhengzhou, 450045, China^d Henan Key Laboratory of Cyberspace Situation Awareness, Zhengzhou, 450001, China

ARTICLE INFO

Article history:

Received 11 April 2023

Received in revised form 25 June 2023

Accepted 26 June 2023

Available online 28 June 2023

Keywords:

Steganalysis

Cover selection

Deep learning

Adaptive filter

ABSTRACT

Image steganalysis aims to detect secret messages embedded in digital images. Nowadays, deep learning-based image steganalysis methods have achieved promising detection performance. However, few existing deep learning-based steganalysis methods take cover selection into consideration. While some networks only utilize fixed filter kernels, which make little use of the learning ability of the network. To alleviate the issue, we propose an image steganalysis method based on cover selection and adaptive filtered residual networks (CSRNet), which can be used for stego images classification. Firstly, we devise a cover selection method, which is specifically divided into two parts. One part is a gray-level co-generation matrix-based (GLCM) multi-scale texture complexity measure to quantify texture complexity. And the other is an inverse power law function-based (IPLF) learning curve, which can contribute to determining the number of selected cover images. Secondly, we design a deep learning steganalysis network based on adaptive filtered kernel and residual learning, which employs the cover-selected images for training, and utilizes the network powerful classification ability. Finally, based on BOSSbase 1.01 and BOWS2, the experimental results show that compared with the classical and state-of-the-art steganalysis methods, the proposed method can not only significantly improve the detection accuracy, but also reduce time-space cost.

© 2023 Elsevier Ltd. All rights reserved.

1. Introduction

Images are widely used in social media [1], which have become one of the main covers for steganography. Steganography can arrive at covert communication by embedding secret messages in cover images and transmitting them through public channels. On the contrary, image steganalysis aims to detect the covert communication by analyzing whether secret messages are embedded in images [2–4]. In a way, steganalysis can be considered a stego image classification technique. The existing typical image steganography methods can be divided into two types: content adaptive steganography methods and deep learning-based steganography methods. The former prefers to embed secret messages into the texture complex region [5–7], which is difficult to model. The latter utilizes deep learning networks to automatically generate steganographic modules [8], among which

the more typical is the generation cover-based deep learning steganography model [9,10].

In recent years, image steganalysis has made tremendous progress, and scholars have devised a series of steganalysis methods, which can be divided into two types: steganalysis methods based on hand-crafted features and deep learning-based steganalysis methods.

Steganalysis methods based on hand-crafted features have achieved promising performance by extracting the latent features of images, which can capture the changes due to the embedding of secret messages. Previous steganalysis methods based on hand-crafted features usually diagnose whether the image contains secret messages, by employing a significant number of statistical features and simple classifiers. Commonly-used statistical features include DCT coefficient features [11], wavelet coefficient features [12], and co-occurrence matrix features [13]. Recently, content adaptive steganography algorithms have brought monumental challenges to steganalysis, due to its rapid development. Accordingly, high-dimensional steganalysis features have emerged. Higher-order statistical features and ensemble classifiers have been used, which are based on the correlation between adjacent pixels. Among them, the SRM features [14] (Spatial

[☆] This article was recommended for publication by Prof. J. Zheng.

* Corresponding author.

E-mail addresses: yuanyuanma821@126.com (Y. Ma), yangzenghao@stu.htu.edu.cn (Z. Yang), lita0116@163.com (T. Li), xulige@stu.htu.edu.cn (L. Xu), qiaoyaqiong@ncwu.edu.cn (Y. Qiao).

Rich Model, recorded as SRM) have achieved great performance. Subsequently, scholars have proposed a series of steganalysis methods, including maxSRM (channel selection SRM) [15] and PSRM (Projected Spatial Rich Model) [16] for the spatial domain. The steganalysis methods used for the JPEG domain include DCTR (DCT Residual) [17], GFR (Gabor Filter Residual) [18], and PHARM (Phase-Aware Projection Model) [19]. Although these steganalysis methods have obtained promising performance, high-dimension brings large time-space cost and bring a heavy burden to the training of classifier.

Deep learning-based steganalysis methods remove the process of manually extracting features and exploit different network structures to learn image characteristics suitable for classification, which achieve superior performance compared to traditional methods [20–32]. Tan and Li et al. [25] firstly employed deep learning into steganalysis, and proposed a deep-stacked convolutional autoencoder network (TanNet) to detect secret message. Therefore, it demonstrated that deep learning can be applied to steganalysis. Inspired by this, Qian et al. [26] proposed a GNCNN (Gaussian-Neuron, CNN) network, which contained a Gaussian activation function, a fixed preprocessing high-pass KV filter with nine layers: one preprocessing layer, five convolutional layers, and three fully connected layers. And the high-pass KV filter could suppress redundant and useless information and improve the signal-to-noise ratio between the cover and stego. Subsequently, Xu et al. [27] proposed Xu-Net on the basis of GNCNN. The network still perpetuated the framework of GNCNN. In addition, ABS layer was added to the first convolutional layer, and BN (Batch Normalization) layer was added for batch processing, and 1×1 convolutional kernel was used to cluster the feature information, which achieved competitive performance. Ye-Net [28] achieved a breakthrough in detection performance, by introducing a truncated linear unit (TLU) activation function and initializing the first layer of convolutional kernels with 30 high-pass filter kernels of the SRM. Boroumand et al. [29] proposed a residual learning-based SRNet. Instead of predefined filters, 64 learnable kernels were used, which could reduce the heuristic design. Residual structure could enhance the learning ability of the network. Experimental results showed that SRNet achieved good detection performance. However, these methods still have limitations, such as heavy computational overhead and lack of data pre-processing operations.

Additionally, despite the type of steganalysis methods, steganographers can always select some suitable images (i.e., perform cover selection) to reduce the probability that the stego images are successfully identified. Scholars have proposed a few methods for cover selection [33–41]. Among them, [35–37,42] point out that images with higher texture complexity had more details, and small modifications can hardly be captured by human vision. Therefore, for uncompressed images, suitable images can be selected for steganography based on the image texture complexity. Whereas cover selection is usually not considered in existing steganalysis methods, it is possible to improve the performance of steganalysis if the steganographers' behavior during cover selection is taken into account [43].

Based on the above mentioned, this paper devises a steganalysis method based on cover selection and adaptive filtered residual network (CSRNet). Firstly, we introduce the cover selection into deep learning-based steganalysis, which is based on the image texture complexity and learning curve. Thus, a residual network based on adaptive filtering is designed to extract features using the powerful learning ability of the network. Finally, the selected texture complex images are put into the proposed network for training, so as to determine whether the images contain secret messages. Through a series of ablation studies and comparative

experiments with other steganalysis networks, we find that cover selection in the process of steganalysis can improve the detection performance and verify the effectiveness of the proposed method. CSRNet achieves outstanding performance in spatial domain steganalysis, which can not only significantly reduce the detection time-space cost, but also achieve excellent detection accuracy.

2. Cover selection

Diversity cover selection methods have merged nowadays, and some scholars suggest using the image texture complexity measure to cover selection. Inspired by this, we introduce cover selection into steganalysis. The GLCM-based multi-scale image texture complexity measure and the inverse power law function-based (IPLF) learning curve are proposed for cover selection.

2.1. GLCM-based image texture complexity measure and selection

The complex texture region of the image has been paid attention to by the existing steganography algorithms. To meet the goal of cover selection, images with a high proportion of complex texture areas are selected as training samples. Thus, in order to quantify the texture complexity of the image, which serves as the foundation for cover selection, the GLCM-based multi-scale image texture complexity measure is devised.

2.1.1. GLCM

The GLCM index is used to quantify the characteristics of image texture [44,45]. We quantify the texture feature of the image by computing the GLCM of the image and certain eigenvalues based on the resulting GLCM. The GLCM can reflect the direction, adjacent interval, change range, and other comprehensive information of the image gray level, and $P(i, j)(i, j = 1, 2, 3, \dots, L)$ is used to represent. Here, $P(i, j)$ represents the probability of the pixel from gray level i to gray level j in the original image, where L represents the gray level of the image. And the corresponding relationship between i and j is satisfied $d = (dx, dy)$, which determines the distance and direction information between two pixels, as shown below:

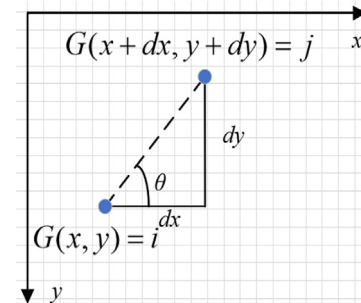


Fig. 1. Relationship between pixel pairs $G(x, y)$ and $G(x + dx, y + dy)$.

In Fig. 1, θ is the scanning angle of GLCM. $G(x, y)$ denotes the gray value of the pixel at the position of (x, y) , and $G(x + dx, y + dy)$ denotes the gray value of the pixel at the position of $(x + dx, y + dy)$. Assuming that traversing all pixels within an image I produces a total of $L * L [G(x, y), G(x + dx, y + dy)]$ binary combinations, count the number of occurrences of all $[G(x, y), G(x + dx, y + dy)]$ pairs and then arrange them into a matrix, assuming that $n [G(x, y), G(x + dx, y + dy)]$ is the number of occurrences of the value pair, so that the total number Δ for all cases is as follows:

$$\Delta = \sum_{i=1}^L \sum_{j=1}^L n[G(x, y), G(x + dx, y + dy)] \quad (1)$$

Each element in the matrix $P_{L \times L}$ is as follows:

$$P[G(x, y), G(x + dx, y + dy)] = \frac{n[G(x, y), G(x + dx, y + dy)]}{\Delta} \quad (2)$$

After the determination of d , a definite GLCM can be generated.

$$P_{L \times L} = \begin{bmatrix} P(1, 1) & P(1, 2) & \cdots & P(1, L) \\ P(2, 1) & P(2, 2) & \cdots & P(2, L) \\ \cdots & \cdots & \cdots & \cdots \\ P(L, 1) & P(L, 2) & \cdots & P(L, L) \end{bmatrix} \quad (3)$$

Thus, the GLCM in one direction is obtained.

As an initial covariance matrix, the GLCM cannot directly be used to describe the texture condition of an image. Thus, it is necessary to gather this matrix's second-order statistics on its foundation. For this purpose, Haralick et al. [44] defined a total of 14 feature parameters of the GLCM. Nevertheless, the above method does not significantly improve the ability for describing the texture and is computationally challenging. Ulaby et al. [46] once more suggested that energy, contrast, entropy, and correlation are all irrelevant.

Based on the above mentioned, we introduce four parameters – Energy, Contrast, Entropy, and Correlation – into the measure, which quantifies the texture complexity of steganographic images, in an effort to more accurately represent the texture complexity of cover or stego and to simplify computation. Here are the specifics:

- (1) Energy(E): The energy is the sum of the squared GLCM values of the cover or stego, also known as the second-order moment. It depicts the degree of texture complexity and gray uniformity of the cover or stego. The calculating formula is as follows:

$$E = \sum_{i=1}^L \sum_{j=1}^L P^2(i, j) \quad (4)$$

where, E represents the energy. i and j represent the gray value of the pixel, and L represents the number of rows or columns of the GLCM.

- (2) Contrast (C): Contrast reflects the covers' or stegos' clarity and texture furrow depth. The contrast value increases as the image becomes crisper and the furrow becomes deeper. The calculating formula is as follows:

$$C = \sum_{i=1}^L \sum_{j=1}^L t^2 P(i, j) \quad (5)$$

where, $t = |i - j|$.

- (3) Entropy (En): Entropy is a measure for gauging how much information a cover or stego has, and texture information is one type of image information. It reflects the texture's complexity, hence the more intricate the texture, the higher the entropy, which is computed as follows:

$$En = - \sum_{i=1}^L \sum_{j=1}^L P(i, j) \log_2 P(i, j) \quad (6)$$

- (4) Correlation (Cor): Correlation measures the degree of similarity of GLCM in a row or column direction. The magnitude of the correlation reflects the strength of the local correlation of the image. If the local correlation of the cover or stego is larger, the correlation value is larger. The calculation formula is as follows:

$$Cor = \frac{1}{\sigma_1 \sigma_2} \sum_{i=1}^L \sum_{j=1}^L ij P(i, j) - \mu_1 \mu_2 \quad (7)$$

where,

$$\begin{cases} \mu_1 = \sum_{i=1}^L i \sum_{j=1}^L P(i, j) \\ \mu_2 = \sum_{j=1}^L j \sum_{i=1}^L P(i, j) \end{cases} \quad (8)$$

$$\begin{cases} \sigma_1 = \sqrt{\sum_{i=1}^L (\mu_1 - i)^2 \sum_{j=1}^L P(i, j)} \\ \sigma_2 = \sqrt{\sum_{j=1}^L (\mu_2 - j)^2 \sum_{i=1}^L P(i, j)} \end{cases} \quad (9)$$

In (8) and (9), μ_1 , μ_2 represent the mean values of GLCM in row and column directions, respectively. σ_1 , σ_2 represent the mean square error of rows and columns, respectively.

2.1.2. Texture complexity measurement

In this section, we introduce the multi-scale fusion (MSF) method to the calculation of integrated texture complexity of the images, since (4)–(7) in Section 2.1.1 calculate 4 texture features under only one direction. This method can more correctly characterize the overall texture complexity of the image. By changing the value of $d = (dx, dy)$ to calculate the texture features under different directions, and using the mean square error to fuse the texture features in different directions, we can reduce the influence of the direction vector on describing the texture complexity. This can contribute to weakening the correlation between texture complexity and direction and better describing the overall texture complexity of the image.

In this paper, we select 4 directions (0° , 45° , 90° , 135°), and generate the GLCM of the image, according to the selected directions. Then, we extract the feature values and perform the multi-scale texture feature fusion based on it. The precise calculation formula is as follows, taking E as an example.

$$\begin{cases} \bar{E} = (E_1 + E_2 + E_3 + E_4)/4 \\ MSE_{E_\vartheta} = (E_\vartheta - \bar{E})^2 \\ MSE_E = \sum_{\vartheta=1}^4 MSE_{E_\vartheta} \\ \alpha_\vartheta = \frac{MSE_{E_\vartheta}}{MSE_E} \\ E = \alpha_1 * E_1 + \alpha_2 * E_2 + \alpha_3 * E_3 + \alpha_4 * E_4 \end{cases} \quad (10)$$

where, E_1 , E_2 , E_3 , E_4 denote the energy of GLCM in 0° , 45° , 90° and 135° directions, respectively. $\bar{E}$ denotes the average energy. MSE_E denotes the mean squared difference of the total energy, and MSE_{E_ϑ} denotes the mean squared difference in each direction. α_1 , α_2 , α_3 , α_4 represent weight values of each direction calculated based on the mean squared error. In order to distribute the weights to each direction, we calculate the sum of the mean squared differences in the 4 directions, i.e. MSE_E , thus we assign the weights α_1 , α_2 , α_3 , α_4 to each direction according to the mean squared difference percentage. Finally, the fusion feature parameter E at multi-scale is obtained.

Similarly, C , En , and Cor can be calculated according to the following equations:

$$\begin{cases} \bar{C} = (C_1 + C_2 + C_3 + C_4)/4 \\ MSE_{C\vartheta} = (C_{\vartheta} - \bar{C})^2 \\ MSE_C = \sum_{\vartheta=1}^4 MSE_{C\vartheta} \\ \alpha_{\vartheta} = \frac{MSE_{C\vartheta}}{MSE_C} \\ C = \alpha_1 * C_1 + \alpha_2 * C_2 + \alpha_3 * C_3 + \alpha_4 * C_4 \end{cases} \quad (11)$$

where, C_1, C_2, C_3, C_4 denote the contrast of GLCM in $0^\circ, 45^\circ, 90^\circ$ and 135° directions, respectively. $\bar{C}$ denotes the average contrast. MSE_C denotes the mean squared difference of the total contrast, and $MSE_{C\vartheta}$ denotes the mean squared difference in each direction.

$$\begin{cases} \bar{En} = (En_1 + En_2 + En_3 + En_4)/4 \\ MSE_{En\vartheta} = (En_{\vartheta} - \bar{En})^2 \\ MSE_{En} = \sum_{\vartheta=1}^4 MSE_{En\vartheta} \\ \alpha_{\vartheta} = \frac{MSE_{En\vartheta}}{MSE_{En}} \\ En = \alpha_1 * En_1 + \alpha_2 * En_2 + \alpha_3 * En_3 + \alpha_4 * En_4 \end{cases} \quad (12)$$

where, En_1, En_2, En_3, En_4 denote the entropy of GLCM in $0^\circ, 45^\circ, 90^\circ$ and 135° directions, respectively. $\bar{En}$ denotes the average entropy. MSE_{En} denotes the mean squared difference of the total entropy, and $MSE_{En\vartheta}$ denotes the mean squared difference in each direction.

$$\begin{cases} \bar{Cor} = (Cor_1 + Cor_2 + Cor_3 + Cor_4)/4 \\ MSE_{Cor\vartheta} = (Cor_{\vartheta} - \bar{Cor})^2 \\ MSE_{Cor} = \sum_{\vartheta=1}^4 MSE_{Cor\vartheta} \\ \alpha_{\vartheta} = \frac{MSE_{Cor\vartheta}}{MSE_{Cor}} \\ Cor = \alpha_1 * Cor_1 + \alpha_2 * Cor_2 + \alpha_3 * Cor_3 + \alpha_4 * Cor_4 \end{cases} \quad (13)$$

where, $Cor_1, Cor_2, Cor_3, Cor_4$ denote the correlation of GLCM in $0^\circ, 45^\circ, 90^\circ$ and 135° directions, respectively. $\bar{Cor}$ denotes the average correlation. MSE_{Cor} denotes the mean squared difference of the total correlation, and $MSE_{Cor\vartheta}$ denotes the mean squared difference in each direction.

Finally, the image texture complexity is derived as follows:

$$f = \omega_1 * E + \omega_2 * C + \omega_3 * En + \omega_4 * Cor \quad (14)$$

where, f is the image texture complexity, $\omega_1, \omega_2, \omega_3$ and ω_4 are the weights assigned to the 4 main feature parameters. We next select 1000 images randomly each time and calculate their respective multi-scale feature parameter value. Then the average of these 1000 images is taken for each feature parameter and recorded. A total of 10 selections are made, finally, the parameters calculated each time are averaged and normalized to obtain the parameters for each feature. The calculation formula of f is as follows:

$$f = \frac{3}{20} * E + \frac{1}{3} * C + \frac{7}{20} * En + \frac{1}{6} * Cor \quad (15)$$

The larger the value of f , the more complex the texture of the image is.

Accordingly, the proposed GLCM-based multi-scale image texture complexity measure algorithm is shown as Algorithm 1.

Algorithm 1 : GLCM-based multi-scale image texture complexity measure algorithm

Input: A standard 256×256 gray-scale image I

Output: E, C, En, Cor and f

- 1: Load in the standard gray-scale image I ;
- 2: Compress the gray level of gray-scale image I to 16 levels;
- 3: **for** $\vartheta = 1$; $\vartheta \leq 4$; $\vartheta++$ **do**
- 4: Construct the GLCM $P_{16 \times 16}$ of I in ϑ direction;
- 5: Calculate the $E_{\vartheta}, C_{\vartheta}, En_{\vartheta}$ and Cor_{ϑ} in the ϑ direction by using Eqs.(4)-(7), respectively
- 6: **end for**
- 7: Calculate the multi-scale fusion feature parameters E, C, En and Cor with Eqs.(10)-(13);
- 8: Calculate the texture complexity f of the image with Eq.(15);
- 9: **Return** E, C, En, Cor and f .

2.2. The images number determination via IPLF-based learning curve

The size of training data and the learning degree of the network are strongly related, although there is not a straightforward linear relationship between them. In other words, it is not always true that bigger data equals higher performance. This is because the larger the data, the more characteristics it contains that are not relevant to classification, which can lead to a decrease in classification performance. Therefore, in order to more effectively select images that are more advantageous for network training, this paper develops an IPLF-based learning curve.

The learning curve is a collection of data points (N_k, Acc_k) , which describes the relationship between the training sample size N_k and the performance Acc_k of a classifier in a given situation, where $k = 1, \dots, m$, m is the total number of samples [47]. The learning curve typically has three phases: phase 1, where classification performance improves quickly as the training set size increases; phase 2, where classification performance slows down and peaks; and phase 3, where classification performance does not improve or only slightly improves as the training set size increases.

The paper is fitted by the following equation to generate the learning curve and afterward determines the number of images to be selected. The IPLF is as follows:

$$Acc = g(N, a, b, c) = a + b * N^c \quad (16)$$

where, the classification accuracy Acc represents as a function of the training set size N , and the parameters a, b, c represent the maximum accuracy, learning rate, and decay rate that the model can achieve, respectively.

We employ a large number of experimental data obtained from four different sizes of training sets (10,000, 20,000, 30,000, 40,000) (see Section 4.2.3 for detailed experiments). In this work, the model is fitted by Eq. (16) to get the relevant parameters, and the unknown parameters are estimated using nonlinear weighted least squares optimization. The model's final learning curve is depicted as follows.

As shown in Fig. 2, N denotes the size of data and Acc denotes the accuracy of the model training. Until the total size of data surpasses 30,000, the accuracy rate rises quickly with the number of samples. And beyond 30,000, expanding data size does not benefit the model's accuracy to advance effectively. The balance between the model's accuracy and the time cost can be best optimized after the number of data hits 30,000. As the size of data reaches 40,000 in the real training process, the accuracy rate starts to decline.

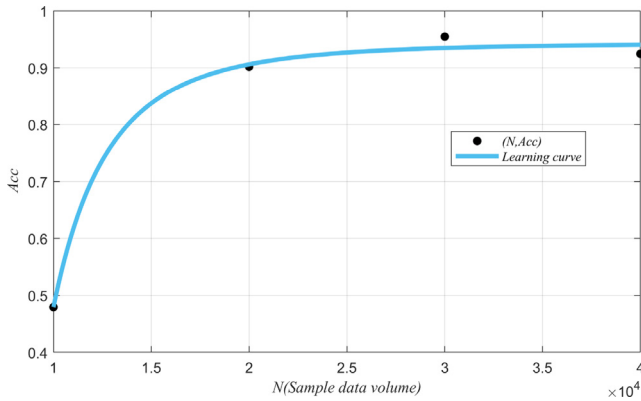


Fig. 2. IPLF-based learning curve.

Hence the fitting results match the experiments and the theoretical size of data needed for model training can be determined. It gives the data size determination some interpretability and evidence as well. Depending on the dataset, sampling strategy, and classification algorithm, the parameter values in this case may vary slightly, but overall fluctuations are not significant.

In summary, the cover selection method devised consists of two parts: the GLCM-based multi-scale image texture complexity measure and the determination of image data based on the IPLF-based learning curve. Section 2.1 utilizes the proposed criterion to measure the texture complexity of images and uses it as the basis for cover selection. Section 2.2 applies the IPLF to fit the learning curve and theoretically determines the number of selected images. The aforementioned methods for cover selection can produce high-quality training data for the network.

3. CSNet for image steganalysis

In this section, we first describe the steganalysis proposed method. Then, explain the overall structure of the network model. Finally, analyze the role of each module and the design principle.

3.1. Overview

Currently, few researchers have applied cover selection to deep learning-based steganalysis, most steganography algorithms tend to embed secret messages into texture complex regions, making it difficult to detect, and network training datasets are too large, causing time cost larger and overfitting. To tackle the above problems, we devise an image steganalysis method based on cover selection and adaptive filtered residual network. Firstly, to address the issue that steganography algorithms are more likely to embed secret messages in texture-complex regions, we calculate the texture complexity of all original image samples based on the GLCM-based multi-scale image texture complexity measure in Section 2.1. Then rank all images accordingly to prepare for the cover selection with complex textures. Secondly, to handle the problems of high training time cost and overfitting of networks due to the vast datasets, we employ the IPLF-based learning curve presented in Section 2.2 to determine the number of selected images with complex textures. Thirdly, according to the number of images specified in Section 2.2, the images are loaded into the designed steganalysis network. The network enriches and extracts the steganographic signals of the images

by adaptive filtering and deep residual learning of the input images. Finally, using the obtained image-related characteristics, it determines whether images contain secret messages. The flow of the proposed method is shown in Fig. 3.

3.2. CSNet

Considering that residual learning works well in convolutional neural networks [48–50] and image filtering is essential for enhancing the network's ability to capture steganographic information, we design an adaptive filtered residual learning network. The structure is shown in Fig. 4. The input of the adaptive filtered residual learning network is 256×256 grayscale images after cover selection, and the output is the judgment result of cover or stego. The network structure consists of three modules: the adaptive filtering module, which can suppress the influence of image content information on the network and strengthen the noise signal formed by steganography; the characteristics extraction module, which can extract the characteristics for classification from the feature map obtained from the adaptive filtering module; and the classification module, which utilizes the features extracted from the characteristics extraction module to determine whether the image contains secret messages. Each module will be described in detail next.

3.2.1. The adaptive filtering module

In general, the high-frequency steganographic messages embedded in cover are weak noise signals, which are significantly influenced by the image content. Therefore, a filtering operation is needed to enhance the noisy signal formed by the secret message before the network training, which can reduce the impact of image content information on detection accuracy. Thus, it can provide a proper initialization to drive the whole network. Compared with direct characteristic extraction or random processing of images, filtering the images enables the network to extract characteristics that are more suitable for classification and improve the performance of the network.

Traditional steganalysis networks preprocess images and suppress visual content information using a fixed high-pass filter kernel known as the KV filter. Thus high-frequency steganographic signals can be boosted as input data for the characteristics extraction module. The KV kernel is as follows:

$$K_{kv} = \frac{1}{12} \begin{pmatrix} -1 & 2 & -2 & 2 & -1 \\ 2 & -6 & 8 & -6 & 2 \\ -2 & 8 & -12 & 8 & -2 \\ 2 & -6 & 8 & -6 & 2 \\ -1 & 2 & -2 & 2 & -1 \end{pmatrix} \quad (17)$$

However, the initialization of the filter kernel is the beginning of network training, not the best scenario. This makes little use of networks' potent learning abilities. Instead of employing a fixed filter kernel, the filter kernel can be updated after initialization. So it also relies on the learning power of network to find the best weights and take full advantage of the deep learning.

Therefore, we design an adaptive filter block to process the image, which uses three convolution kernels that can be learned to filter the image by training. It can more efficiently suppress the content information of the image, to strengthen the noise signal caused by the embedded secret message, which can provide a reliable and effective initialization for subsequent characteristic extraction. We also design Stage0 as the activation block to expand the filtered feature maps, so that the residual network can perform characteristic extraction on it.

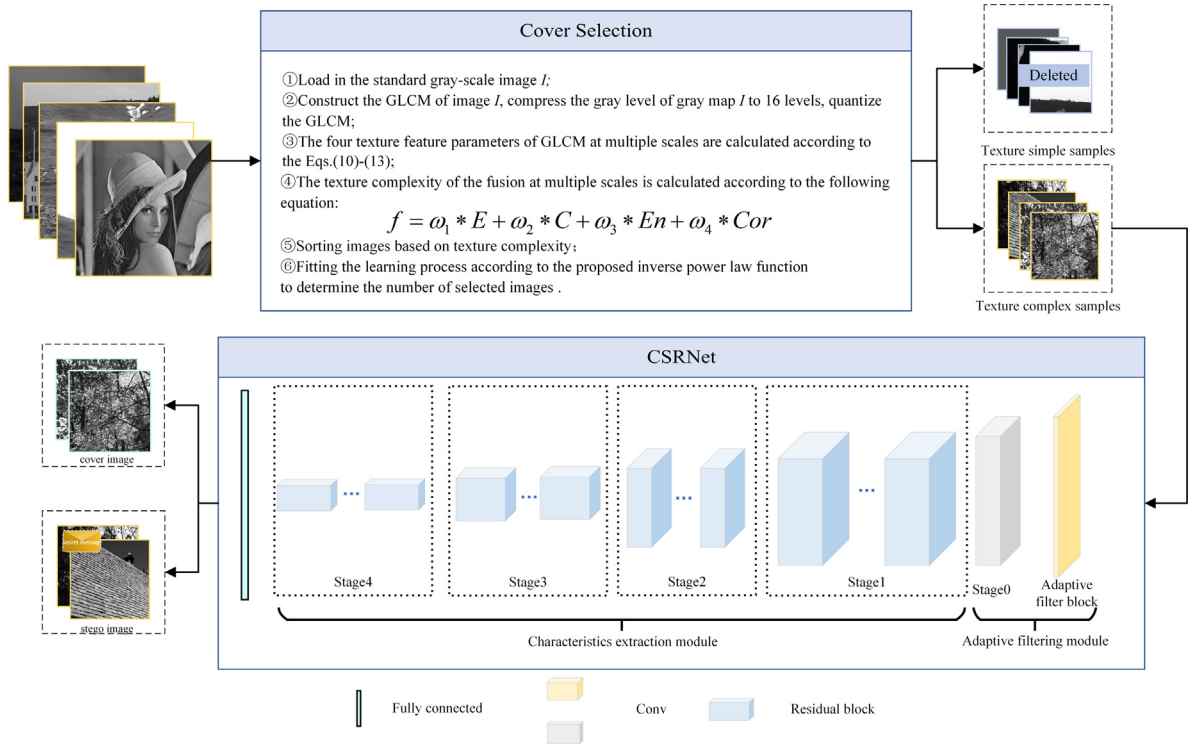


Fig. 3. Flowchart of steganalysis method for spatial domain images based on cover selection and adaptive filtered residual network.

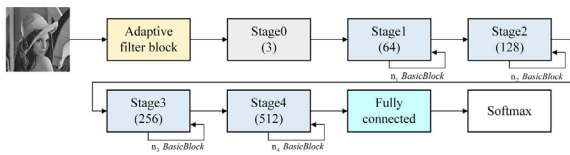


Fig. 4. Structure of CSRNet.

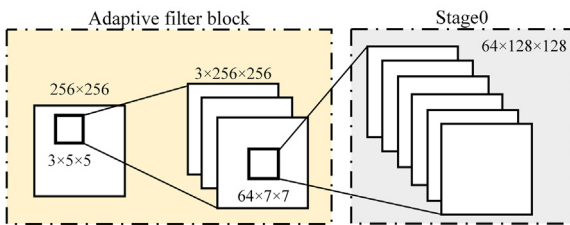


Fig. 5. Details of adaptive filtering module.

The specific structure of the adaptive filtering module is shown in Fig. 5, which consists of two parts: the adaptive filter block and the Stage0 activation block. Firstly, the 256×256 grayscale input images are passed through three 5×5 convolution kernels in adaptive filter block to obtain $3 \times 256 \times 256$ feature maps. Secondly, we obtain the $64 \times 128 \times 128$ feature maps through $64 \times 7 \times 7$ convolution kernels in Stage0, which are followed by Batch Normalization, activation, and maximum pooling operations, and subsequently perform characteristic extraction. In the adaptive filter block, the three filter kernels are initialized with the KV high-pass filter kernel of Eq. (17). Thus, the useless

information can be suppressed and the steganography signal can be strengthened.

3.2.2. The characteristics extraction module

The characteristics extraction module is responsible for extracting the classification characteristics from the feature maps for the classification module to discriminate the results. Residual learning is originally proposed by He [51], and we are inspired to design a network based on residual learning.

The Stage1-Stage4 part in Fig. 4 is the characteristics extraction module. In this module, the features are extracted using residual learning, and each feature map is downsampled for the input image. Here, residual learning uses two operation blocks to process the data: the residual learning block (as structured in Fig. 6(a)) and the dimension increasing block (as structured in Fig. 6(b)). Each residual learning block has the same input and output feature dimensions, and has two convolutional layers. The size of each convolution kernel is 3×3 . The number of convolution kernels is the same as the number of the feature maps, and each convolution layer is followed by a Batch Normalization layer and a Relu activation layer. The dimension increasing block is a block in which the output feature dimension is twice as large as the input feature dimension. In each block, there is a shortcut path that connects the input and output of each block and acts as the same mapping for the "adding" operation. The difference between the dimension increasing block and the residual learning block is that there is a 1×1 convolution kernel in the shortcut path of the dimension increasing block to expand the dimension of the original features on the shortcut path, which enables the output feature dimension to be doubled.

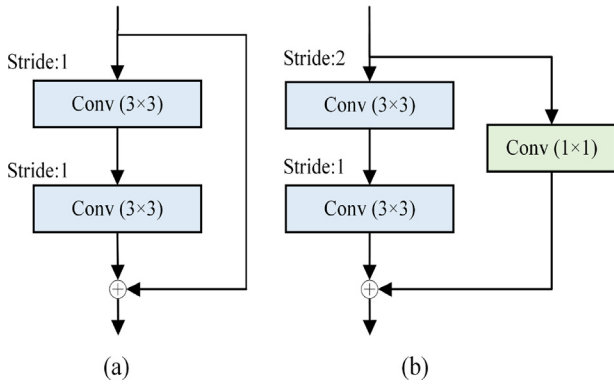


Fig. 6. Residual operation block (left side (a) is the residual learning block, right side (b) is the dimension increase block).

Usually, each Stage is connected with a dimension increase block for dimension expansion, followed by several residual learning blocks for multi-level residual learning. The overall network is composed of several Stages, each of which can convolve several times to extract deeper characteristics, allowing the network to capture tiny noise signals. Finally, the image is converted into several feature maps, which are sent to the classification module.

Due to the emergence of residual learning, the network can be deepened without limitation, but deepening the network without limitation does not lead to an unlimited improvement in network performance, while it causes an increase in time-space cost. Simultaneously, practical steganalysis also pursues a certain degree of timeliness. Therefore, the network designed in this paper, set $[n_1, n_2, n_3, n_4] = [2, 2, 2, 2]$ (see Fig. 4) with a total of 19 layers network, which makes the network achieve promising performance and reduces the time-space cost.

3.2.3. The classification module

The classification module consists of a fully connected layer and a Softmax layer. Before the fully connected layer, the two-dimensional feature maps obtained in the characteristics extraction module are averaged and pooled to integrate characteristics. At the fully connected layer, the two-dimensional feature map obtained by the characteristics extraction module is converted into a one-dimensional feature vector. The Softmax layer is responsible for converting the one-dimensional vector passed from the fully connected layer into probability and outputting the final judgment result.

4. Experiments

This section conducts extensive experiments to verify the effectiveness of the proposed CSNet. Specifically, Section 4.1 describes some of the experimental setups conducted for the experiments, and Section 4.2 presents a series of related experiments.

4.1. Experimental setup

The datasets for the experiments are BOSSbase 1.01 [52] and BOWS2 [53], both of which are 10,000 images of 512×512 , which contain various types of images such as life, attractions, and architecture. BOSSbase 1.01 is a dedicated dataset obtained from seven different types of digital cameras for steganalysis contests. And BOWS2 dataset is used for watermarking contests and has similar characteristic distribution and content as the

BOSSbase 1.01 dataset. In the experiments, due to the GPU computing power and time limitation, all images are cropped into four 256×256 images that do not overlap each other. Then the secret message is embedded in the cover to get the stego.

In the adaptive filtering module, the three convolution kernels are initialized as KV kernels. And the learning rate in the adaptive filter block is set to 0.0001. In the characteristics extraction module and the classification module, the weight matrix $\mathbf{W}$ of the network layer is initialized to a Gaussian distribution, with mean equal to 0 and standard deviation equal to 0.01. And the biases vector $\mathbf{B}$ is initialized to 0. The learning rate of $\mathbf{W}$ and $\mathbf{B}$ is initially set to 0.001 and 0.0001 after 50 epochs. Each batch of data is set to 20 (10 covers and 10 stegos). Parameters of the proposed network are learned by minimizing the softmax loss function. The optimizer uses SGD, and the number of training epochs is set to 100. The experiments were run on a 16.0 GB RAM, 64-bit OS, x64-based Intel(R) Core (TM) i7-10750H CPU @ 2.60 GHz 2.59 GHz processor, and NVIDIA GeForce RTX 2060 GPU.

We use the error detection rate (Err) to evaluate the effectiveness of the steganalysis network:

$$Err = 1 - \frac{N_{RC} + N_{RS}}{N_C + N_S} \quad (18)$$

where, N_C and N_S denote the number of cover and stego, respectively. N_{RC} and N_{RS} denote the number of samples of correctly classified cover and stego, respectively.

4.2. Experimental results comparison and analysis

In order to verify the effectiveness of the proposed method, this section is validated by a series of ablation studies and comparison experiments.

4.2.1. Validation of image texture complexity

In this section, we design experiments to verify the validity and correctness of the proposed algorithm in Section 2.2, which measures the texture complexity of images. Using Eqs. (4)–(15), the texture complexity of each image is calculated; then, we rank these images by the calculation results. Simulation experiments are conducted under the MATLAB experimental platform, and the results of the experimental part are shown in the following figures.

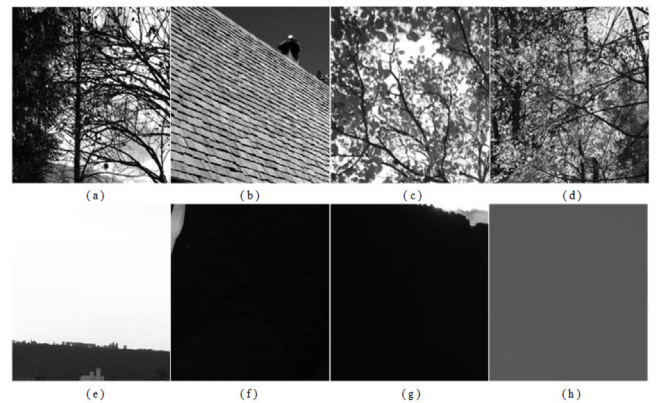


Fig. 7. Cover selection-ed images obtained by calculating texture complexity. As shown in Fig. 7, (a) (b) (c) (d) are the images with higher ranking based on texture complexity and larger quantized texture

complexity after sorting, and (e) (f) (g) (h) are the images with lower ranking based on texture complexity and smaller quantized texture complexity. This result is consistent with the visual observation of the image texture complexity. Thus, it can be verified that the mathematical relationship established by the GLCM-based multi-scale image texture complexity measure proposed in this paper holds, and also verify the correctness and rationality of the proposed algorithm visually.

4.2.2. Comparison with other textural features

Commonly used methods for texture feature extraction are gray level co-occurrence matrix (GLCM), Tamura texture features [54], etc. To demonstrate the superior performance of the GLCM-based texture complexity measure proposed in this paper, we compare the measure used with the Tamura texture feature.

Tamura texture features are based on a psychological study of human visual perception of textures with six attributes: Coarseness, Contrast, Directionality, Line-Likeness, Regularity, and Roughness. The first three parameters are commonly used in practical applications. In this paper, these three parameters are selected and then the parameters for each of their features are determined using the same method, resulting in a Tamura-based texture complexity measure. The Euclidean distance is used to verify the rotational invariance of both measures and is calculated as follows:

$$D = \sqrt{\gamma_1^2 - \gamma_2^2} \quad (19)$$

where, γ_1 is the texture complexity before rotation, γ_2 is the texture complexity after rotation, and D is the Euclidean distance between the texture complexity before and after rotation. A smaller Euclidean distance indicates higher robustness of the algorithm.

We randomly chose 100 images and rotated them (rotation angles of 30°, 45°, 60°, and 75°). Using the GLCM-based measure and the Tamura texture feature-based measure to calculate their texture complexity, respectively. And then calculating the Euclidean distance at each angle, the experimental results are shown in Table 1:

Table 1

Comparison of the Euclidean distance between the GLCM-based measure and the Tamura texture feature-based measure for different rotation angles.

Angle of rotation	30°	45°	60°	75°
GLCM	0.3039	0.3515	0.3036	0.1812
Tamura	1.7363	3.4767	2.4717	0.6398

Table 1 shows that the Euclidean distance of GLCM is much smaller than that of Tamura, so the GLCM-based measure employed in this paper performs much better. In addition, we compared the time cost of the two algorithms to compute the complexity of these 100 images as shown in Table 2:

Table 2

Comparison of the time cost of the GLCM-based measure and the Tamura texture feature-based measure.

Algorithms	Time cost (s)
GLCM-based	71.3
Tamura-based	238.7

From Table 2, it can be observed that the GLCM takes 71.3s, significantly better than the Tamura-based measure. In a conclusion, the texture complexity measure developed in this paper has significant advantages in terms of both robustness and time complexity.

4.2.3. The result of cover selection

To verify whether cover selection with high texture complexity is helpful for steganalysis, we design ablation study. The number of cover selection is determined to be 30,000 based on the experimental results by fitting the learning curve proposed in Section 2.2. This experiment is conducted based on BOSSbase 1.01 dataset, and the experiment is performed based on the sorted images obtained in Section 4.2.1, which selects 20,000, 30,000, and 40,000 pairs of images and input them to CSNet for training. The results are shown in Table 3.

Table 3

Comparison of *Err* before and after cover selection (on S-UNIWARD steganography algorithm at 0.4 bpp).

The number of images	20k pairs images	30k pairs images	40k pairs images
Random	0.1416	0.0779	0.0653
Cover selection	0.1080	0.0551	0.0653

According to Table 3, when selecting only 20,000 pairs of images, the *Err* of random selection method is 14.16%, and the *Err* after cover selection is 10.80%, which is reduced by 3.36%. When selecting 30,000 pairs of images, the *Err* of the random selection method is 7.79%, and the *Err* after cover selection is 5.51%, which is reduced by 2.28%. Without cover selection, 40,000 original images are directly taken for training, and the result is 6.53%. When the number of selected images is the same as the fitting result in Section 2.2, which is 30,000, the training effect of the network is the best.

The experimental results show that employing cover selection for steganalysis can improve the detection accuracy, reduce the time cost and improve the timeliness of the method.

4.2.4. Comparison with and without adaptive filtering

As mentioned previously, adaptive filtering is employed to further improve the detection accuracy of the network. In order to verify whether the adaptive kernel is helpful in improving the detection accuracy, a validation experiment of the adaptive filtering module is designed. The experimental data is obtained from 30,000 pairs of cover-selected images. The stegos are obtained by S-UNIWARD at 0.2 bpp. The experimental results are shown in Table 4:

Table 4

Err for the network with and without adaptive filter block on S-UNIWARD at 0.2 bpp.

Configuration	Adaptive kernel	Fixed kernel
<i>Err</i>	0.1601	0.1869

Table 4 shows the *Err* of CSNet with adaptive and fixed filtering kernels. The experimental results show that using the adaptive filtering kernel to suppress the image content information can significantly reduce the *Err* of the network.

4.2.5. Comparison with other deep learning methods

We use three mainstream content adaptive steganography algorithms S-UNIWARD, WOW, and HUGO to generate stegos, which are carried out on the same BOSSbase 1.01 dataset. And the payloads are set to 0.2, 0.3, 0.4, and 0.5, respectively. The steganography algorithms are all carried out in MATLAB environment. The proposed method is compared with Wu-Net [32], SRNet [29], Zhu-Net [31], and GBARS-Net [30], which are four classical and state-of-the-art deep learning steganalysis methods. The image pairs composed of cover and stego were divided into training set and test set according to the ratio of 3:1. In the proposed method, 30,000 images are selected by cover selection, then the training set and test set are randomly divided into 3:1 ratio. The experimental results are as follows:

Table 5

Err comparison of Wu-Net, SRNet, Zhu-Net, GBRAS-Net and CSRNet (against the three steganography algorithms S-UNIWARD, WOW and HUGO with payloads of 0.2 to 0.5, and the best performance is shown in bold).

Steganography algorithm	Payload	Wu-Net [32] <i>Err</i>	SRNet [29] <i>Err</i>	Zhu-Net [31] <i>Err</i>	GBRAS-Net [30] <i>Err</i>	CSRNet <i>Err</i>
S-UNIWARD	0.2	0.3202	0.1692	0.2650	0.2430	0.1601
	0.3	0.2672	0.1270	0.2057	0.1722	0.0894
	0.4	0.2257	0.0922	0.1832	0.1273	0.0421
	0.5	0.1926	0.0713	0.1622	0.0892	0.0393
WOW	0.2	0.3232	0.2452	0.2365	0.1920	0.1663
	0.3	0.2695	0.1985	0.1832	0.1460	0.0849
	0.4	0.0085	0.1562	0.1500	0.1090	0.0663
	0.5	0.1883	0.1285	0.1266	0.0719	0.0432
HUGO	0.2	0.3480	0.2892	0.2700	0.2598	0.1451
	0.3	0.2892	0.2337	0.2202	0.2025	0.1061
	0.4	0.2520	0.2052	0.1752	0.1633	0.0470
	0.5	0.1972	0.1722	0.1320	0.1294	0.0364

As shown in Table 5, for S-UNIWARD steganography algorithm with different payloads, the *Err* of CSRNet is lower 4.99% to 8.52% than that of GBRAS-Net, 0.91% to 5.01% lower than SRNet. And the overall *Err* is lower about 12% than that of the network Zhu-Net. For the WOW steganography algorithm with different payloads, CSRNet also has a significant reduction in the *Err*, compared with the other four steganalysis networks. And the largest increase is achieved at 0.3 payload, with a *Err* reduction of 6.11%. For HUGO steganography algorithm with different payloads, the *Err* of CSRNet is significantly decreased. At the 0.4 payload, the *Err* is 11.63% lower than that of GBRAS-Net. In summary, regardless of the embedding methods and payloads, CSRNet has a significant improvement in accuracy compared with other networks.

CSRNet adds an adaptive filtering module, which can make the network capture the small noise signal brought by steganographic embedding. In order to effectively extract the features for classification, the overall residual learning is used to construct the network architecture. And the characteristics are extracted through the deep residual network. Experimental results show that CSRNet achieves an excellent performance.

In addition, to verify that CSRNet works equally well for images with simple texture, we also conducted image generalizability experiments. In this experiment, we tested the network with a cover selection of the remaining 10,000 images with simple textures as the test set. The stegos are obtained using the S-UNIWARD at 0.2–0.5 bpp. The experimental results are shown in the table:

Table 6

Err of CSRNet tested on simple texture images (on S-UNIWARD at 0.2–0.5 bpp).

Payload (bpp)	0.2	0.3	0.4	0.5
<i>Err</i>	0.1781	0.1132	0.0498	0.0397

Table 6 shows the *Err* of CSRNet for simple texture images (on S-UNIWARD at 0.2–0.5 bpp). Compared to the experimental data in Table 5, the *Err* improves slightly. The change is smallest at 0.5 bpp, where little change occurs. The *Err* improvement is greatest at 0.3 bpp, but is still the most accurate of several detection networks. Therefore, CSRNet has a strong generalization capability and excellent detection accuracy in both textured complex and textured simple images.

4.2.6. Experiment on stego-source mismatch

The generalization performance of the model determines whether the model can be applied to real-life scenarios, so the generalization performance of the model is also an index of the success for the model. In the practical application of steganalysis, there may be cases that the steganographic algorithms used for

training images and detection images are not the same, which is called stego-source mismatch. We design stego-source mismatch experiments for the above situation. The network is trained and tested with images generated by different steganography algorithms.

Table 7 shows the *Err* of the proposed method and three other methods, SRNet, Zhu-Net, and GBRAS-Net, in the stego-source mismatch experiments. It can be observed from Table 7 that the network trained by S-UNIWARD algorithm has the best detection accuracy, and can also keep a stable detection accuracy when it is used to detect the images, which are embedded by the steganography algorithms of WOW and HUGO, with the maximum fluctuation of only 1.42%. When testing the steganography images embedded by HUGO algorithm, the generalization ability is the strongest, and the *Err* is the closest, with 4.5%. The network trained by the WOW algorithm has a slight lack of generalization ability, and the *Err* fluctuated greatly when it was applied to the images embedded by the HUGO steganography algorithm, which was 6.86%. The network trained by the HUGO algorithm also keeps promising performance of 7.19% and 7.62%, when detecting stego embedded by S-UNIWARD and WOW steganography algorithms.

The experimental results show that the *Err* of CSRNet is lower than that of the other three models when the stego-sources are mismatched. The *Err* fluctuates in a small range when the stego-sources are different in the training and testing images, and the generalization performance of the model is promising.

4.2.7. Data augmentation experiment

In deep learning, the number of training data is usually required to be sufficient. To some extent, the larger the number of training data, the better the training model is. And the generalization capacity of the model is stronger. In order to further verify the generalization ability of the model, we conduct data expansion across datasets. So that it can expand the training set for the purpose of verifying the generalization ability of the model. In this experiment, the data is obtained from two databases, BOSSbase 1.01 and BOWS2, and a total of 80,000 pairs of images (a pair of images includes a cover and a stego) are obtained. Then divide them into a training set and a test set in the ratio of 3:1. In the proposed method, 20,000 images with complex textures are selected from each of the two databases, totaling 40,000 pairs of images for training and testing.

Table 8 shows the comparison of the *Err* of SRNet, GBRAS-Net, Zhu-Net, and CSRNet trained on different training data for the embedding algorithm WOW at a payload of 0.2. The experimental results show that using BOSSbase 1.01 + BOWS2 can achieve

Table 7

Err of SRNet, Zhu-Net, GBRAS-Net and the proposed method in the stego-source mismatch experiment (all steganography algorithms under this experiment are carried out at a payload of 0.4).

Steganography for training	Steganography for testing	Steganalysis networks			
		SRNet [29]	Zhu-Net [31]	GBRAS-Net [30]	CSRNet
S-UNIWARD	S-UNIWARD	0.0922	0.1890	0.1273	0.0421
	WOW	0.2372	0.1912	0.1550	0.0563
	HUGO	0.2247	0.2847	0.3145	0.0450
WOW	S-UNIWARD	0.2292	0.2570	0.1283	0.0710
	WOW	0.1562	0.1500	0.1090	0.0663
	HUGO	0.3937	0.3987	0.3332	0.1349
HUGO	S-UNIWARD	0.3515	0.3797	0.3685	0.0719
	WOW	0.3607	0.3652	0.4143	0.0762
	HUGO	0.2052	0.1752	0.1633	0.0470

Table 8

Comparison of steganalysis *Err* using CSRNet, SRNet, GBRAS-Net, and Zhu-Net against WOW at 0.2 bpp with training data augmented with BOWS2.

Algorithms	BOSSbase 1.01	BOSSbase 1.01+BOWS2
SRNet [29]	0.2452	0.1642
Zhu-Net [31]	0.2365	0.2122
GBRAS-Net [30]	0.1920	0.1730
CSRNet	0.1663	0.1024

better network performance and enhance the generalization ability of the network, compared with using only BOSSbase 1.01. The CSRNet obtains the best detection performance than other steganalysis networks, and using BOSSbase 1.01 + BOWS2 can be an *Err* reduction of 6.41% compared to using only BOSSbase 1.01.

Table 9

Comparison of steganalysis *Err* using CSRNet, SRNet, GBRAS-Net and Zhu-Net against S-UNIWARD at 0.2 bpp and 0.4 bpp, with training data augmented with BOWS2.

Algorithms	Payload = 0.2 bpp	Payload = 0.4 bpp
SRNet [29]	0.1247	0.0545
Zhu-Net [31]	0.2390	0.1347
GBRAS-Net [30]	0.2210	0.1270
CSRNet	0.0893	0.0283

Table 9 lists the *Err* of CSRNet and the other three steganalysis networks in the stego embedded by S-UNIWARD at 0.2 bpp and 0.4 bpp. Among them, the CSRNet has the lowest *Err*, which can reach 2.83% at a payload of 0.4 and 8.93% at a payload of 0.2, which has the best performance compared with these other steganalysis networks.

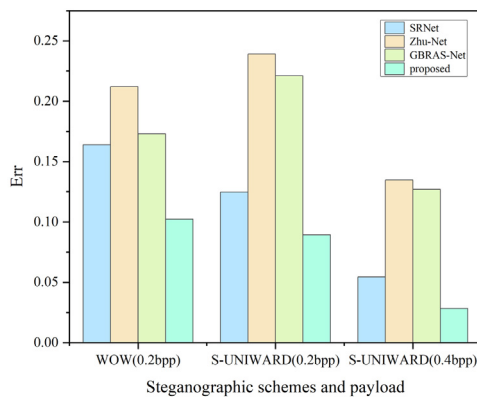


Fig. 8. Comparison of steganalysis *Err* of the four steganalysis methods against WOW and S-UNIWARD algorithms at 0.2 bpp and 0.4 bpp. All experiments are based on BOSSbase 1.01+BOWS2.

As shown in Fig. 8, the results of data augmentation experiments for four steganalysis networks embedded by the WOW algorithm at 0.2 bpp and the S-UNIWARD algorithm at 0.2 bpp and 0.4 bpp. In this experiment, the *Err* of CSRNet is lower than other steganalysis networks. For S-UNIWARD algorithm at 0.4 bpp, it can reach 2.83% *Err*. In this experiment, CSRNet has the lowest *Err*, and achieves the best generation performance.

4.2.8. Comparison of model complexity

In convolutional neural networks, network complexity is an important factor to estimate the performance of the model. And the complexity includes model complexity as well as time complexity. In this section, we compare the number of parameters and the time cost for training of CSRNet, SRNet, Zhu-Net, and GBRAS-Net, and the results are obtained experimentally on the same devices. The results are listed in Table 10.

Table 10

Number of parameters, training time for SRNet, Zhu-Net, GBRAS-Net and CSRNet (training time is the time from the start of model training to the completion of training).

Algorithms	Number of parameters(10^6)	Training time
SRNet [29]	47.7	42 h
Zhu-Net [31]	28.7	12.45 h
GBRAS-Net [30]	1.70	15.5 h
CSRNet(30,000 Images)	11.7	24.3 h
CSRNet(40,000 Images)	11.7	32.7 h

As shown in Table 10, GBRAS-Net has the smallest number of parameters. SRNet has the largest number of parameters. CSRNet has 11.7×10^6 parameters, which has lower model complexity than SRNet and Zhu-Net. SRNet is based on the powerful learning ability of neural networks to extract noise residuals without a priori knowledge support, while the network structure is more complex, so the parameters are the largest. Zhu-Net and GBRAS-Net directly use the 30 high-pass filters in SRM to initialize the filtering convolution kernel. Therefore, Zhu-Net and GBRAS-Net have fewer parameters and shorter training time. CSRNet relies less on prior knowledge than Zhu-Net. It extracts noise based on the fitting ability of the neural network with little prior knowledge. As a result, it has fewer parameters and takes longer to train than Zhu-Net. In CSRNet, the difference in the number of images does not cause changes in the network frame, so the number of parameters remains the same. While the network training time is shorter, as the images are selected. CSRNet employs adaptive filtering kernels and deep residual learning to extract image characteristics for classification. The number of parameters is small and the training time does not take particularly long, so CSRNet has a decent overall performance in model complexity.

5. Conclusion

In this paper, we advise CSRNet, which achieves promising performance in the spatial domain. Compared with existing steganalysis methods, CSRNet achieves significant improvement. The

keys to the performance improvement of CSRNet are as follows: (1) cover selection is introduced into the deep learning-based steganalysis method. We propose a GLCM-based multi-scale texture complexity measure to quantify the texture complexity as the basis for cover selection. The IPLF-based learning curve is also proposed to determine the specific number of cover selection. Finally, cover selection is performed based on the above two methods. (2) An adaptive filtered residual network is proposed. Employing the adaptive filtering module can reduce the influence of the image content information, and strengthen the weak noise signal brought by the embedded secret message. Utilizing deep residual learning, the deep potential information of the image can be learned, which is more conducive to capturing the noisy signal. The experimental results show that the cover selection method achieves excellent performance in steganalysis. The proposed CSRNet has remarkable advancement in performance and more generalization ability compared with other methods. In the future works, we will propose different cover selection methods for steganalysis, and optimize the network to reduce the number of parameters with guaranteed accuracy.

CRediT authorship contribution statement

Yuanyuan Ma: Data curation. **Zenghao Yang:** Methodology, Writing – original draft, Writing – review & editing. **Tao Li:** Software. **Lige Xu:** Writing – review & editing. **Yaqiong Qiao:** Conceptualization, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (62002103, 62272163), the Outstanding Youth Program of Henan Province (222300420058), the Henan Province Science Foundation for Youths (222300420230), the Key Scientific Research Project of Henan Provincial Higher Education under Grant (22B520013), the Key Scientific and Technological Project of Henan Province under Grant (222102210169).

References

- [1] Lin X, Huang Q, Huang W, Tan X, Ma L. Single image deraining via detail-guided efficient channel attention network. *Comput Graph* 2021;97(2). <http://dx.doi.org/10.1016/j.cag.2021.04.014>.
- [2] Fridrich J. *Steganography in digital media: Principles, algorithms, and applications*. Cambridge, U.K.: Cambridge Univ. Press; 2009.
- [3] Wang H, Wang S. Cyber warfare: steganography vs. steganalysis. *Commun ACM* 2004;47(10):76–82. <http://dx.doi.org/10.1145/1022594.1022597>.
- [4] Guo L, Ni J, Yun Q. Uniform embedding for efficient JPEG steganography. *IEEE Trans Inf Forensics Secur* 2014;9(5):814–25. <http://dx.doi.org/10.1109/TIFS.2014.2312817>.
- [5] Sedighi V, Cogramme R, Fridrich J. Content-adaptive steganography by minimizing statistical detectability. *IEEE Trans Inf Forensics Secur* 2016;11(2):221–34. <http://dx.doi.org/10.1109/TIFS.2015.2486744>.
- [6] Holub V, Fridrich J, Denemark T. Universal distortion function for steganography in an arbitrary domain. *EURASIP J Inf Secur* 2014;2014(1):1–13. <http://dx.doi.org/10.1186/1687-417X-2014-1>.
- [7] Yu K, Chen L, Wang Y, Han J, Zhang L. Reversible data hiding in encrypted images for coding channel based on adaptive steganography. *IET Image Process* 2020;14(16):4229–39. <http://dx.doi.org/10.1049/iet-ipr.2020.1105>.
- [8] Goodfellow I, Pouget A, Mirza M, Xu B, Warde D, Ozair S, Courville A, Bengio Y. Generative adversarial nets. In: *Proceedings of the 2014 international conference on neural information processing systems*. Montreal, Canada; 2014. p. 2672–80.
- [9] Vpikonskiy D, Borisenko B, Burnaev E. Generative adversarial networks for image steganography. In: *Proceedings of the open review conference on learning representations*. Puerto Rico, USA; 2016.
- [10] Shi H, Dong J, Wang W, Zhang X. SSGAN: secure steganography based on generative adversarial networks. In: *Proceedings of the pacific rim conference on multimedia*. Harbin, China; 2017. p. 534–44.
- [11] Fridrich J. Feature-based steganalysis for JPEG images and its implications for future design of steganographic schemes. In: *Proc. 6th int. workshop inf. hiding. Lecture notes in computer science*, vol. 3200, 2004, p. 67–81. http://dx.doi.org/10.1007/978-3-540-30114-1_6.
- [12] Luo X, Liu F, Lian S, Yang C, Gritzalis S. On the typical statistic features for image blind steganalysis. *IEEE J Sel Areas Commun* 2011;29(7):1404–22. <http://dx.doi.org/10.1109/JSA.2011.110807>.
- [13] Yong W, Liu J, Zhang W. Blind JPEG steganalysis based on correlations of DCT coefficients in multi-directions and calibrations. In: *International conference on multimedia information networking & security*. IEEE; 2009.
- [14] Fridrich J, Kodovsky J. Rich models for steganalysis of digital images. *IEEE Trans Inf Forensics Secur* 2012;7(3):868–82. <http://dx.doi.org/10.1109/TIFS.2012.2190402>.
- [15] Denemark T, Sedighi V, Holub V, Cogramme R, Fridrich J. Selection-channel-aware rich model for steganalysis of digital images. In: *IEEE international workshop on information forensics and security*. Atlanta, Georgia; 2014.
- [16] Holub V, Fridrich J. Random projections of residuals for digital image steganalysis. *IEEE Trans Inf Forensics Secur* 2013;8(12):1996–2006. <http://dx.doi.org/10.1109/TIFS.2013.2286682>.
- [17] Holub V, Fridrich J. Low-complexity features for JPEG steganalysis using undecimated DCT. *IEEE Trans Inf Forensics Secur* 2015;10(2):219–28. <http://dx.doi.org/10.1109/TIFS.2014.2364918>.
- [18] Song X, Liu F, Yang C, Luo X, Zhang Y. Steganalysis of adaptive JPEG steganography using 2D Gabor filters. In: *Proc. 3rd ACM workshop inf. hiding multimedia secur.* Portland, OR, USA; 2015. p. 15–23.
- [19] Holub V, Fridrich J. Phase-aware projection model for steganalysis of JPEG images. *Proc SPIE* 2015;9409:94090–T–1–94090T–11.
- [20] Qian Y, Dong J, Wang W, Tan T. Feature learning for steganalysis using convolutional neural networks. *Multimedia Tools Appl* 2018;77(15):19633–57.
- [21] You W, Zhang H, Zhao X. A siamese CNN for image steganalysis. *IEEE Trans Inf Forensics Secur* 2020;1. <http://dx.doi.org/10.1109/TIFS.2020.3013204>.
- [22] Ying Z, Ge Z, Liu L. Research on image steganography analysis based on deep learning. *J Vis Commun Image Represent* 2019;60:266–75. <http://dx.doi.org/10.1016/j.jvcir.2019.02.034>.
- [23] Xu G. Deep convolutional neural network to detect J-UNIWARD. In: *Proc. 5th ACM workshop inf. hiding multimedia security (IH & MMSec)*. Philadelphia, PA, USA: Drexel Univ.; 2017. p. 67–73.
- [24] Singhal A, Bedi P. Multi-class blind steganalysis using deep residual networks. *Multimedia Tools Appl* 2021;80(9):13931–56. <http://dx.doi.org/10.1007/s11042-020-10353-2>.
- [25] Tan S, Li B. Stacked convolutional auto-encoders for steganalysis of digital images. In: *Signal and information processing association annual summit and conference (APSIPA)*, 2014 Asia-Pacific. IEEE; 2014. p. 1–4.
- [26] Qian Y, Jing D, Wei W, Tan T. Deep learning for steganalysis via convolutional neural networks. *Proc SPIE - Int Soc Opt Eng* 2015;9409. <http://dx.doi.org/10.1117/12.2083479>, 94090J–94090J–10.
- [27] Xu G, Wu H, Shi Y. Structural design of convolutional neural networks for steganalysis. *IEEE Signal Process Lett* 2016;23(5):708–12. <http://dx.doi.org/10.1109/LSP.2016.2548421>.
- [28] Jian Y, Ni J, Yang Y. Deep learning hierarchical representations for image steganalysis. *IEEE Trans Inf Forensics Secur* 2017;12(11):2545–57. <http://dx.doi.org/10.1109/TIFS.2017.2710946>.
- [29] Boroum M, Chen M, Fridrich J. Deep residual network for steganalysis of digital images. *IEEE Trans Inf Forensics Secur* 2019;14(5):1181–93. <http://dx.doi.org/10.1109/TIFS.2018.2871749>.
- [30] Rubio A, Grisales J, Soto R, Ramos-Pollán, Jàcome ABB. GBRAS-Net: A convolutional neural network architecture for spatial image steganalysis. *IEEE Access* 2021;9:14340–50. <http://dx.doi.org/10.1109/ACCESS.2021.3052494>.
- [31] Zhang R, Zhu F, Liu J, Liu G. Depth-wise separable convolutions and multi-level pooling for an efficient spatial CNN-based steganalysis. *IEEE Trans Inf Forensics Secur* 2020;15:1138–50. <http://dx.doi.org/10.1109/ACCESS.2021.3052494>.
- [32] Wu S, Zhong S, Liu Y. Deep residual learning for image steganalysis. *Multimedia Tools Appl* 2018;77(9):10437–53. <http://dx.doi.org/10.1007/s11042-017-4440-4>.
- [33] Kharrazi M, Sencar H, Memon N. Cover selection for steganographic embedding. In: *IEEE international conference on image processing*. 2006. p. 117–20.

- [34] Evsutin O, Kokurina A, Meshcheryakov R. Approach to the selection of the best cover image for information embedding in JPEG images based on the principles of the optimality. *J Decis Syst* 2018;27(S1):256–64. <http://dx.doi.org/10.1080/12460125.2018.1460163>.
- [35] Sajedi H, Jamzad M. Cover selection steganography method based on similarity of image blocks. In: *IEEE 8th international conference on computer and information technology*. IEEE; 2008, p. 379–84.
- [36] Sajedi H, Jamzad M. Using contourlet transform and cover selection for secure steganography. *Int J Inf Secur* 2010;9(5):337–52.
- [37] Subhedar M, Mankar V. Curvelet transform and cover selection for secure steganography. *Multimedia Tools Appl* 2018;77(7):8115–38.
- [38] Wu S, Liu Y, Zhong S, Liu Y. What makes the stego image undetectable? In: *Proceedings of the 7th international conference on internet multimedia computing and service*. ACM; 2015, p. 47.
- [39] Wang Z, Zhang X, Yin Z. Joint cover-selection and payload-allocation by steganographic distortion optimization. *IEEE Signal Process Lett* 2018;25(10):1530–4. <http://dx.doi.org/10.1109/LSP.2018.2865888>.
- [40] Wang Z, Zhang X. Secure cover selection for steganography. *IEEE Access* 2019;7:57857–67. <http://dx.doi.org/10.1109/ACCESS.2019.2914226>.
- [41] Wang Z, Zhang X, Qian Z. Practical cover selection for steganography. *IEEE Signal Process Lett* 2020;27(1):71–5. <http://dx.doi.org/10.1109/LSP.2019.2956416>.
- [42] Wu J, Liu C, Li B. Texture-aware and structure-preserving superpixel segmentation. *Comput Graph* 2020. <http://dx.doi.org/10.1016/j.cag.2020.12.002>.
- [43] Wang Z, Li S, Zhang X. Towards improved steganalysis: When cover selection is used in steganography. *IEEE Access* 2019;7:168914–21. <http://dx.doi.org/10.1109/ACCESS.2019.2955113>.
- [44] Haralick R, Shanmugam K, Dinstein I. Textural features for image classification. *Stud Media Commun* 1973. <http://dx.doi.org/10.1109/TSMC.1973.4309314>.
- [45] Chen Y, Duan J, Zhu Y, Qian F, Xiao B. Research on image complexity based on texture feature. *ChineeOptics* 2015;8(3):407–14.
- [46] Ulaby F, Kouyate F, Brisco B, Williams T. Textural information in SAR images. *IEEE Trans Geosci Remote Sens* 1986;24(2):235–45. <http://dx.doi.org/10.1109/TGRS.1986.289643>.
- [47] Figueroa R, Zeng-Treitler Q, Kandula S, Long H. Predicting sample size required for classification performance. *BMC Med Inform Decis Mak* 2012;12(1):8. <http://dx.doi.org/10.1186/1472-6947-12-8>.
- [48] Zhang H, Pu Y, Nie R, Xu D, Qian W. Multi-modal image synthesis combining content-style adaptive normalization and attentive normalization. *Comput Graph* 2021;98. <http://dx.doi.org/10.1016/j.cag.2021.04.030>.
- [49] Olivier N, Baert K, Danieau F, Avril Q. FaceTuneGAN: Face autoencoder for convolutional expression transfer using neural generative adversarial networks. 2021. <http://dx.doi.org/10.48550/arXiv.2112.00532>, arXiv e-prints.
- [50] Li C, Tan Y, Chen W, Luo X, Li F. ANU-Net: Attention-based Nested U-Net to exploit full resolution features for medical image segmentation. *Comput Graph* 2020;90. <http://dx.doi.org/10.1016/j.cag.2020.05.003>.
- [51] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In: *2016 IEEE conference on computer vision and pattern recognition*. CVPR, Las Vegas, NV, USA; 2016, p. 770–8. <http://dx.doi.org/10.1109/CVPR.2016.90>.
- [52] Bas P, Filler T, Pevný T. Break our steganographic system – the ins and outs of organizing BOSS. In: *Filler T, Pevný T, Ker A, Craver S, editors. Information hiding, 13th international conference. Lecture notes in computer science*, vol. 6958, Prague, Czech Republic: Springer Berlin Heidelberg; 2011, p. 59–70.
- [53] Bas P, Furon T. BOWS-2. 2008. <http://bows2.ec-lille.fr>.
- [54] Tamura Hideyuki, Mori Shunji, Yamawaki Takashi. Textural features corresponding to visual perception. *IEEE Trans Syst Man Cybern* 1978. <http://dx.doi.org/10.1109/TSMC.1978.4309999>.